DOCUMENTATION TO THE CDCL PROJECT

TEEMU LUNDSTRÖM

This document describes theoretical ideas and implementation details related to the project in this repository, which implements a CDCL SAT solver. It explains the specific implementations and ideas used in the project, and not the general idea of the CDCL algorithm or SAT solvers in general. This document was written mainly for personal learning purposes.

In Section 1, we give a quick overview of the whole application. In Section 2, we fix some notation related to propositional logic used in this document. In Section 3, we explain the Tseitin transformation and in Section 4 we give an overview of the features and implementation of the CDCL algorithm, including a proof of correctness. In Section 5, we quickly explain how we express a sudoku problem with a CNF formula.

1. Overview

Figure 1 shows a flowchart of the application when the input is a string representing a propositional formula. There the input is fed into a parser that checks if the string is a well-formed formula. If the parser outputs a parsing or scanning error, the program outputs an error message to the user and stops. Otherwise the parser constructs a parsing tree of the formula. The parsing tree is then used to construct the Tseitin set (a set of clauses) of the given formula (see Section 3). Before the Tseitin set is constructed, a variable (a label) is given to each subformula, which is done by recursively visiting each node of the syntax tree. After the Tseitin set is constructed, preprocessing is applied to the set of clauses. Namely, each clause containing the atomic formula $\top$ (see Section 2) is removed, and the atomic formula $\bot$ is removed from every clause. In addition, we remove possible duplicate literals from clauses, remove duplicate clauses, and remove all tautological clauses (clauses containing a literal and its negation). After the preprocessing, we then start the CDCL algorithm whose output is then either a satisfying assignment or a resolution proof showing unsatisfiability.

Figure 2 shows a flowchart of the application when the input is either a DIMACS CNF file, or a sudoku file. In both cases, from the given file a set of clauses is constructed. Then, a similar preprocessing is done as above: we remove duplicate literals from clauses, remove duplicate clauses and remove all tautological clauses. We then run the CDCL algorithm which gives the same output as before.

Remark 1. When the program reads a .cnf file or a sudoku file, the program does not currently check if the file follows the assumed format, but blindly assumes it does. If the file does not follow the assumed format, errors might occur.

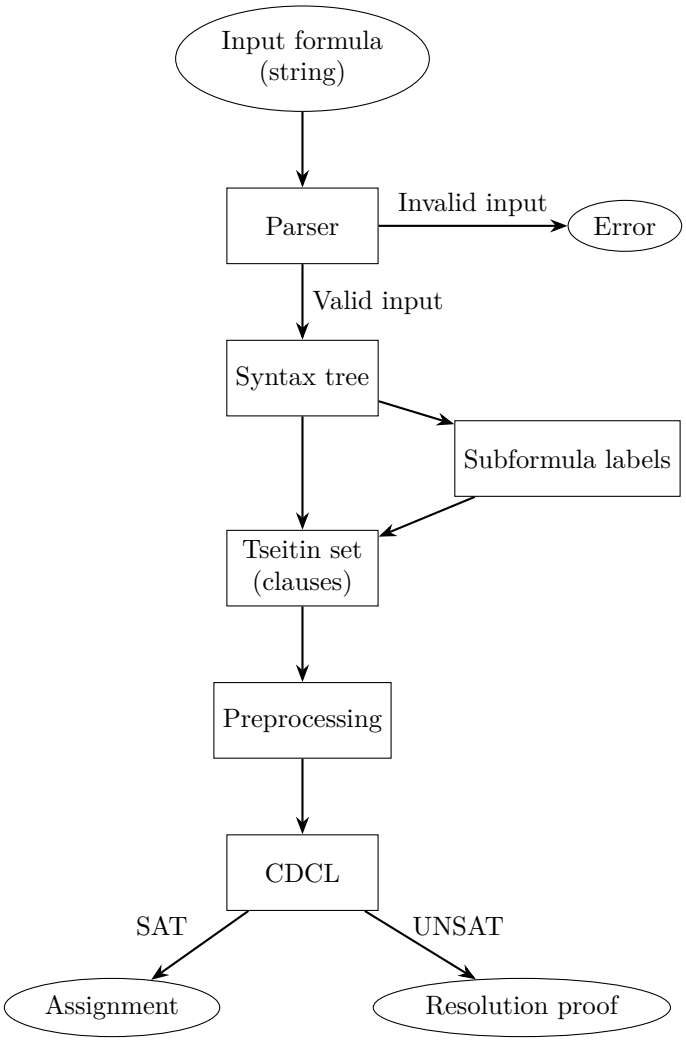

FIGURE 1. Flowchart of the program when the input is a formula.

2. Propositional Logic

We start by explaining some of the notation and terminology from propositional logic used in this document, which we assume the reader is mostly familiar with.

In our propositional formulas we use connectives $\neg$ (negation), $\vee$ (disjunction), $\wedge$ (conjunction), $\rightarrow$ (implication), $\leftrightarrow$ (equivalence). We also have atomic formulas $\top$ (true) and $\bot$ (false). The set of variables appearing in a propositional formula A is denoted by $\mathrm{Vars}(A)$.

A *truth assignment* for a formula A is a function $v\colon \mathrm{Vars}(A) \rightarrow \{0,1\}$ where 1 represents true and 0 represents false. We think of the four connectives as taking input 0/1 and giving output 0/1 in the usual way. For example, $0 \wedge 1 = 0$, $1 \vee 1 = 1$, $0 \rightarrow 1 = 1$, and so on. A truth assignment then extends to a formula in the usual inductive way. Specifically, we set $v(\neg B) = \neg v(B)$ and $v(B_1 \Box B_2) = v(B_1) \Box v(B_2)$

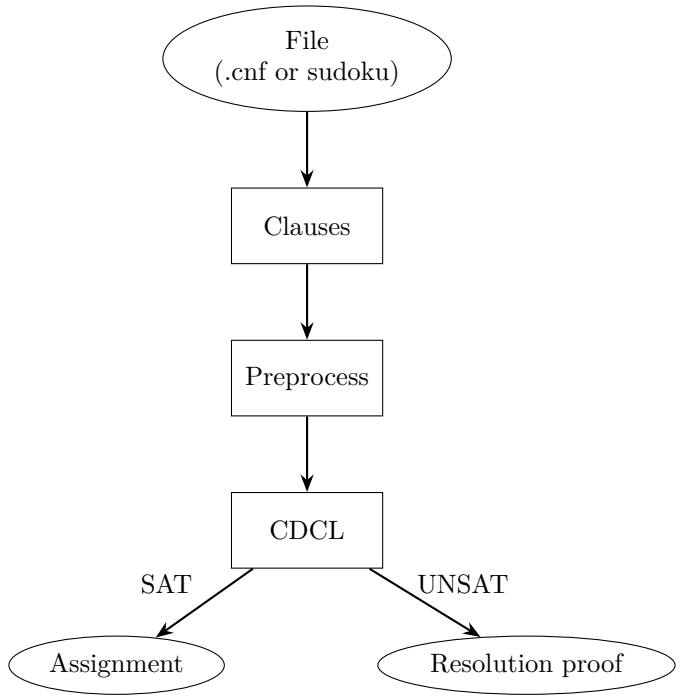

FIGURE 2. Flowchart of the program when the input is a DIMACS CNF file or a sudoku file.

for formulas B, B_1, B_2 and binary connective $\square \in \{\wedge, \vee, \rightarrow, \leftrightarrow\}$. For any truth assignment v we set $v(\top) = 1$ and $v(\bot) = 0$.

For example, if $v(a) = 0$ and $v(b) = 0$, we may compute

$$v(\neg a \rightarrow b) = v(\neg a) \rightarrow v(b) = \neg v(a) \rightarrow v(b) = \neg 0 \rightarrow 0 = 1 \rightarrow 0 = 0.$$

A *literal* is a variable or its negation. A *clause* is a (possibly empty) disjunction of literals. A CNF (Conjunctive Normal Form) formula is a (possibly empty) conjunction of clauses. We usually think of clauses as sets of literals and CNF formulas as sets of clauses. In our notation we assume double negations are automatically removed. For example, if ℓ is $\neg x$ for some variable x then by $\neg \ell$ we mean x.

For CNF formulas we will consider *partial truth assignments* which are truth assignments where only some of the variables have been assigned a truth value. Given a partial truth assignment, a clause is true if at least one of its literals is true, and a CNF formula is true if all of its clauses are true. A clause C is said to be a *unit* (with respect to some partial assignment) if one literal $\ell \in C$ is undefined and all other literals in C are false. In this case we say ℓ is *implied*, or more accurately, that ℓ is *implied by* C. A clause is said to be *falsified* if all of its literals are false.

For clauses C and D and literal ℓ we denote the resolution of them with respect to ℓ by $\mathrm{Res}(C, D, \ell)$. That is, $\mathrm{Res}(C, D, \ell) = (C \cup D) \setminus \{\ell, \neg \ell\}$.

3. Tseitin set

When the user gives a propositional formula A as input, we compute a CNF formula that is equisatisfiable with A. Specifically, we construct the *Tseitin set*, which is defined as follows.

Definition 1 (Tseitin set)**.** Let A be a propositional formula. For each subformula B of A we let

$$\begin{cases} q_B \text{ be a new variable if } B \text{ is not atomic} \\ q_B = B \text{ if } B \text{ is atomic i.e. a variable or } \top \text{ or } \bot. \end{cases}$$

For each subformula B of A that is not atomic we add the following clauses to the set $\mathcal{T}_A$.

(1) If $B = \neg C$ we add

$$\{q_B, q_C\}, \{\neg q_B, \neg q_C\}.$$

(2) If $B = C_1 \vee C_2$ we add

$$\{\neg q_B, q_{C_1}, q_{C_2}\}, \{q_B, \neg q_{C_1}\}, \{q_B, \neg q_{C_2}\}.$$

(3) If $B = C_1 \wedge C_2$ we add

$$\{\neg q_B, q_{C_1}\}, \{\neg q_B, q_{C_2}\}, \{q_B, \neg q_{C_1}, \neg q_{C_2}\}.$$

(4) If $B = C_1 \to C_2$ we add

$$\{\neg q_B, \neg q_{C_1}, q_{C_2}\}, \{q_B, q_{C_1}\}, \{q_B, \neg q_{C_2}\}.$$

(5) If $B = C_1 \leftrightarrow C_2$ we add

$$\{\neg q_B, q_{C_1}, \neg q_{C_2}\}, \{\neg q_B, \neg q_{C_1}, q_{C_2}\}, \{q_B, q_{C_1}, q_{C_2}\}, \{q_B, \neg q_{C_1}, \neg q_{C_2}\}.$$

Lastly, we add $\{q_A\}$ to $\mathcal{T}_A$. The set $\mathcal{T}_A$ is called the *Tseitin* set of A.

Thus the Tseitin set $\mathcal{T}_A$ is a set of clauses so we may think of $\mathcal{T}_A$ as a CNF formula. Note that $\mathrm{Vars}(A) \subseteq \mathrm{Vars}(\mathcal{T}_A)$. As the next theorem shows, the Tseitin set is satisfiable if and only if the original formula is satisfiable.

Theorem 1. *Let A be a formula and let $\mathcal{T}_A$ be its Tseitin set. Then A is satisfiable if and only if $\mathcal{T}_A$ is satisfiable. If $v\colon \mathrm{Vars}(\mathcal{T}_A) \to \{0,1\}$ satisfies $\mathcal{T}_A$ then v satisfies A.*

Proof. Suppose A is satisfiable. Let $v\colon \mathrm{Vars}(A) \to \{0,1\}$ be such that $v(A) = 1$. We define $v'\colon \mathrm{Vars}(\mathcal{T}_A) \to \{0,1\}$ as follows. For each subformula B of A

(1) if B is atomic we let $v'(q_B) \coloneqq v(B)$,
(2) if $B = \neg C$ and $v'(q_C)$ is defined, we let $v'(q_B) \coloneqq \neg v'(q_C)$,
(3) if $B = C_1 \Box C_2$ where $\Box \in \{\wedge, \vee, \to, \leftrightarrow\}$ and $v'(q_{C_1})$ and $v'(q_{C_2})$ are defined, we let $v'(q_B) \coloneqq v'(q_{C_1}) \Box v'(q_{C_2})$.

With induction it is easy to show that $v'(q_B) = v(B)$ for all subformulas B of A. With this, it is straightforward to check that all the clauses we added to $\mathcal{T}_A$ are satisfied. Thus $v'(\mathcal{T}_A) = 1$ and hence $\mathcal{T}_A$ is satisfiable.

Then suppose $v\colon \mathrm{Vars}(\mathcal{T}_A) \to \{0,1\}$ is a truth assignment where $v(\mathcal{T}_A) = 1$. We show that for all subformulas B of A we have $v(q_B) = v(B)$. For atomic formulas B we have $v(q_B) = v(B)$ since $q_B = B$. For the inductive step we only show the case for disjunction since all the other cases are similar. Suppose $B = C_1 \vee C_2$

where $v(q_{C_i}) = v(C_i)$. By looking at all possible values for $v(q_B), v(q_{C_1}), v(q_{C_2})$ we get the table

$v(q_B)$	$v(q_{C_1})$	$v(q_{C_2})$	$\{\neg q_B, q_{C_1}, q_{C_2}\}, \{q_B, \neg q_{C_1}\}, \{q_B, \neg q_{C_2}\}$
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Since the clauses $\{\neg q_B, q_{C_1}, q_{C_2}\}, \{q_B, \neg q_{C_1}\}, \{q_B, \neg q_{C_2}\}$ are all true (by assumption), the only possible rows are the ones where the last column has a 1. On all such rows we have

$$v(q_B) = v(q_{C_1}) \vee v(q_{C_2}) = v(C_1) \vee v(C_2) = v(C_1 \vee C_2) = v(B).$$

We conclude that $v(q_B) = v(B)$ for all subformulas B of A. Now in particular with $B = A$ we have $v(q_A) = v(A)$ and since v satisfies the clause $\{q_A\}$ we have $v(A) = 1$. □

In our implementation, we recursively visit each subformula using the syntax tree of the formula, giving labels to each subformula. In the implementation, the new variables q_B will be integers (with negative integers representing negations), except for atomic formulas $\top$ and $\bot$, which we leave as such. Thus the clauses in the Tseitin set will consist of integers and the two symbols $\top$ and $\bot$. Before we give these clauses for the CDCL algorithm, we remove all clauses with a $\top$ in them and remove $\bot$ from every clause. Clearly the clauses obtained this way are satisfiable if and only if the original Tseitin set is satisfiable.

4. CDCL

In this section we explain how we implemented the CDCL algorithm. Section 4.1 gives some further terminology and Section 4.2 explains the concept of two-watched literals. In Section 4.3 we give an overview of the algorithm. In Section 4.4 we sketch a proof of correctness of the algorithm, and in Section 4.5 we explain how we build a resolution proof during the algorithm. Most of our terminology and notation comes from these lecture notes.

4.1. Trail, annotated literals, etc.

We record all variable assignments in a list called the *trail*. The trail will consist of *annotated literals*, which are symbols of the form ℓ^R where ℓ is a literal and R is a clause, and ℓ^{dec} where ℓ is a literal and dec is a special symbol. Annotated literals of the form ℓ^R are called *implied literals* and the clause R is called the *reason clause* for ℓ^R (also known as the *antecedent*). Annotated literals of the form ℓ^{dec} are called *decision literals*.

The idea is that if a literal ℓ is implied by a clause R we must make ℓ true and thus we add ℓ^R to end the trail. If we make a new decision and set a literal ℓ to true, we add ℓ^{dec} to the end trail. This way we record not only which literals are true but also why they are true, that is, are they implied by a clause or are they true because of a decision. The corresponding truth assignment can be read from

the trail: each literal in the trail is assigned true, their negations are assigned false, and variables not appearing in the trail are undefined.

The *level* (also called *decision level*) of an annotated literal ℓ^r in the trail is the number of decision literals appearing in the trail before (and including) ℓ^r. We denote this by $\text{lev}(\ell^r)$ or by just $\text{lev}(\ell)$. If ℓ appears in the trail (and hence $\neg\ell$ does not), we set $\text{lev}(\neg\ell) = \text{lev}(\ell)$. The position (i.e. index) of a literal ℓ appearing in the trail is denoted by $\text{pos}(\ell)$. The *level of a trail* is the number of decision literals in it. We often speak of the *current level* which is the level of the trail at that point in the algorithm.

For example, a trail could look like $(\ell_1^{C_1}, \ell_2^{C_2}, \ell_3^{\text{dec}}, \ell_4^{\text{dec}}, \ell_5^{C_5})$ for some literals ℓ_i and clauses C_i. The sequence of levels would be $(0, 0, 1, 2, 2)$ and the sequence of positions would be $(1, 2, 3, 4, 5)$. The level of this trail is 2.

4.2. **Two-watched literals.** In our implementation of the CDCL we implement a so called two-watched literal scheme that greatly speeds up unit propagation.

The idea behind two-watched literals is that every (non-singleton) clause will have two literals that are *watched*. The aim is to always choose the watched literals so that both are non-false (i.e. true or undefined) or, if not possible, that one is true and the other is false (if possible). A clause with two non-false literals cannot be false or a unit, and neither can a clause with a true literal. When we add a literal ℓ to the trail, we then only need to check those clauses where $\neg\ell$ is watched, as those are the only ones that might have become units or false.

When a watched literal w turns false, we choose the new watched literal (if we can) from the literals in the same clause in the following way.

(i) If there is a non-false unwatched literal ℓ in the clause, we choose ℓ as the new watched literal and we do not watch w anymore.
(ii) Otherwise (all unwatched literals are false), if the other watched literal w' is true, we keep w as the watched literal.
(iii) Otherwise, if w' is undefined we know w' is an implied literal, which we then add to the trail.
(iv) Otherwise, w' is false and we know we have a falsified clause.

For singleton clauses, both watched literals will be equal to the same literal (we may think of two pointers pointing to the same literal).

Using this scheme, every time we add a literal ℓ to the trail we only need to check clauses where $\neg\ell$ is watched and update the watched literals there accordingly. This removes the need to go over all the clauses to find unit or falsified clauses, greatly speeding up unit propagation.

4.3. **Overview of the algorithm.** Here we give an overview of how our implementation works. Algorithm 1 shows pseudo code for the whole CDCL algorithm. The algorithm uses procedures INITIALIZE (Procedure 2), UNIT-PROPAGATION (Procedure 3), CLAUSE-LEARNING (Procedure 4) and BACKTRACK (Procedure 5).

In our implementation we will have a (first-in first-out) queue `to_propagate` where we store all the annotated literals that we want to add to the trail. More specifically, whenever we make a new decision, we (randomly) choose an unassigned literal ℓ and add ℓ^{dec} to the end of `to_propagate`. Whenever we have a literal ℓ implied by a clause R we add ℓ^R to the end of `to_propagate`. During unit propagation we pop these literals from `to_propagate` one by one, add them to the trail and update the watched literals accordingly.

The input to the algorithm will be a set $\mathcal{C}$ of clauses representing a CNF formula φ. We assume all the procedures here have access to the trail, information about watched literals and to the set $\mathcal{C}$ of clauses, even though we do not give these explicitly as inputs.

The first step is to run the procedure INITIALIZE (Procedure 2) which sets the watched literals for each clause in $\mathcal{C}$. For each singleton clause we set the only literal as the watched literal. All singleton clauses are units with respect to the empty trail and thus for each singleton $\{\ell\} \in \mathcal{C}$ we need to add $\ell^{\{\ell\}}$ to `to_propagate`. For clauses of size ≥ 2 we choose any two literals as the two watched literals. [1] If INITIALIZE finds an empty clause we can report that φ is not satisfiable.

We then start the main loop in Algorithm 1. At the beginning of each iteration we first execute UNIT-PROPAGATION (Procedure 3). There we pop an annotated literal ℓ^r from `to_propagate` and add it to the trail (if not already there). After that, we go through all clauses C that have $\neg\ell$ as a watched literal and update the watched literals accordingly. If we find that C is false we report that C is falsified and exit UNIT-PROPAGATION. If we find a C implies a literal w we add w^C to `to_propagate`. We continue this as long as `to_propagate` is non-empty.

If UNIT-PROPAGATION did not falsify any clauses, we then check if all variables are assigned. If yes, we have have found a satisfying assignment which we can then return. If not, we choose an unassigned literal ℓ (essentially randomly), add ℓ^{dec} to `to_propagate` and start the next iteration. To further optimize the algorithm, different heuristics could be used when selecting the unassigned literal (such as VSIDS).

If UNIT-PROPAGATION ends with a falsified a clause and the current level is 0, we know that φ cannot be satisfiable as literals on level 0 are logical consequences of φ. We can thus report that φ is not satisfiable and exit the algorithm.

If UNIT-PROPAGATION ends with a falsified clause and the current level is not 0, we do clause learning and backtracking. For this we first execute the procedure CLAUSE-LEARNING (Procedure 4). There, we continuously do resolutions to find the 1-UIP clause, which has exactly one literal on the current level. After CLAUSE-LEARNING we add the learned clause C_{learned} to $\mathcal{C}$. The next step is then to do backtracking with the procedure BACKTRACK (Procedure 5). There, we split into two cases depending on the size of the learned clause C_{learned}.

(i) If C_{learned} is a singleton we set its only literal as the watched literal and backtrack to level 0. After backtracking, C_{learned} will then be a unit and thus we must add its only literal ℓ to `to_propagate`. As there are no other units after backtracking (see Section 4.4) we set `to_propagate` $:= (\ell^{C_{\text{learned}}})$, a queue with one element.

(ii) If C_{learned} has at least two literals, we watch literals ℓ_1 and ℓ_2 where ℓ_1 has the highest level and ℓ_2 has the second highest level. We then backtrack to level $\text{lev}(\ell_2)$. After backtracking, ℓ_1 will be implied by C_{learned} and it will be the only implied literal. Hence we set `to_propagate` $:= (\ell_1^{C_{\text{learned}}})$.

After backtracking, we then start the next iteration. We continue this loop until we have found a falsified clause at level 0, or until we have assigned every variable, in which case we have found a satisfying assignment.

[1] In our implementation, as preprocessing we remove all tautological clauses (clauses that have both ℓ and $\neg\ell$) as well as duplicate literals from all clauses. Hence the two watched literals will have different variables.

Algorithm 1: CDCL

```
input  : A CNF formula φ given as a set C of clauses
output: SAT and a satisfying assignment or UNSAT
INITIALIZE
while True do
    UNIT-PROPAGATION
    if a clause is falsified then
        if current level is 0 then
            return UNSAT
        let C be a falsified clause
        set C_learned := CLAUSE-LEARNING(C)
        add C_learned to C
        BACKTRACK(C_learned)
    else
        if all variables are assigned then
            return SAT and the current assignment
        choose an unassigned literal ℓ
        set to_propagate := (ℓ^dec)          // a queue with one element
```

Procedure 2: INITIALIZE

```
set to_propagate := ∅
for C ∈ C do
    if |C| = 0 then
        report that φ is UNSAT
    else if |C| = 1 then
        let ℓ be the only literal in C
        set ℓ to be the only watched literal in C
        add ℓ^C to to_propagate
    else
        pick two literals as the watched literals
```

4.4. **Correctness.** Here we sketch a proof of correctness of our implementation. We only give a sketch of a proof, leaving some of the details to the reader.

There are two important invariants related to watched literals that we want to maintain throughout the algorithm.

Invariant 1. If one watched literal is false and the other is not true, then all unwatched literals are false.

Invariant 2. If a watched literal w_1 is true and the other watched literal w_2 is false, then $\mathrm{lev}(w_1) \leq \mathrm{lev}(w_2)$.

Invariant 1 guarantees that a clause is falsified if and only if both of its watched literals are false, and that a (non-singleton) clause is a unit if and only if one watched literal is false and the other is undefined.

Procedure 3: UNIT-PROPAGATION

```
while to_propagate is non-empty do
    pop ℓ^a from to_propagate
    if ℓ is in the trail then
        continue the while loop
    add ℓ^a to the trail
    for C in the set of clauses where ¬ℓ is watched do
        if there exists ℓ' ∈ C that is not watched and not false then
            set ℓ' as the new watched literal
        else
            let w be the other watched literal
            if w is false then
                report that C is falsified and exit
            if w is true then
                continue the for loop
            if w is undefined then
                add w^C to to_propagate
```

Procedure 4: CLAUSE-LEARNING

```
input  : Clause C ∈ 𝒞 that is currently false
output: The learned (1-UIP) clause C_learned
set C_learned = C
let d be the current level
while C_learned contains at least two level d literals do
    let ℓ maximize {pos(¬ℓ) | ℓ ∈ C_learned}
    let R be the reason clause for ¬ℓ
    let C_learned := Res(C_learned, R, ℓ)
return C_learned
```

Procedure 5: BACKTRACK

```
input  : The learned clause C_learned
if |C_learned| = 1 then
    set the only literal ℓ as the watched literal
    remove all literals from the trail of level > 0
    set to_propagate := (ℓ^{C_learned})          // a queue with one element
else
    let ℓ_1 ∈ C_learned have the highest level          // the current level
    let ℓ_2 ∈ C_learned have the second highest level
    set ℓ_1 and ℓ_2 as the watched literals
    remove all literals of level > lev(ℓ_2) from the trail
    set to_propagate := (ℓ_1^{C_learned})          // a queue with one element
```

Invariant 2 guarantees that if during backtracking a true watched literal turns undefined, the other watched literal cannot then be false. This helps one show that after backtracking, Invariant 1 continues to hold for all clauses.

We start by looking at unit propagation.

Lemma 1. *The while loop in* UNIT-PROPAGATION *eventually stops.*

Proof. When we pop a literal from `to_propagate` we immediately make it true if it is not already on the trail. The only time we add a literal to `to_propagate` is when that literal is undefined (line 16). Since we never set a literal from 0/1 to undefined during UNIT-PROPAGATION, every time we add a literal to `to_propagate`, the set of literals that we can later add to `to_propagate` strictly decreases. Thus eventually `to_propagate` will be empty or we reach line 12 at some point. □

Lemma 2. *After every time* UNIT-PROPAGATION *finishes, all singleton clauses in $\mathcal{C}$ are true at level 0 (unless we have a conflict at level 0).*

Proof. One can check that at no point in Algorithm 1 do we remove literals at level 0 form the trail. Hence if a singleton clause is true at level 0 at some point, it will stay true until the end of the algorithm.

In the procedure INITIALIZATION, for every singleton $\{\ell\} \in \mathcal{C}$ we put ℓ into `to_propagate`. During the first run of UNIT-PROPAGATION we then add all these literals to the trail (unless we find a falsified clause, in which case we have a conflict at level 0). Suppose we learn a new singleton clause $C_{\text{learned}} = \{\ell\}$ in CLAUSE-LEARNING which we then add to $\mathcal{C}$. In the procedure BACKTRACK, we will then backtrack to level 0 and set `to_propagate` $= (\ell^{C_{\text{learned}}})$. During next unit propagation we will then set ℓ to true at level 0. Hence all singleton clauses continue to be true at level 0. □

As we will see later, there are three cases what `to_propagate` will look like at the beginning of the procedure UNIT-PROPAGATION.

(TP1) `to_propagate` $= (\ell_1^{\{\ell_1\}}, \ldots, \ell_k^{\{\ell_k\}})$ where the ℓ_i are the only implied literals. This happens after initialization and after learning a singleton clause and backtracking to level 0 (in which case $k = 1$).

(TP2) `to_propagate` $= (\ell^R)$ where R is a non-singleton clause that implies ℓ. This happens after learning a non-singleton clause. Here ℓ will be the only implied literal.

(TP3) `to_propagate` $= (\ell^{\text{dec}})$. This happens after making a new decision. In this case there are no implied literals.

Remark 2. As we never add decision literals to `to_propagate` during UNIT PROPAGATION, all literals that are assigned during UNIT-PROPAGATION will have the same level, which is the current level at that point.

Lemma 3. *If the following hold at the beginning of* UNIT-PROPAGATION*, they will hold at the beginning of each iteration of the while loop in* UNIT-PROPAGATION*.*

(i) There are no falsified clauses.

(ii) Every clause satisfies Invariants 1 and 2.

(iii) If one watched literal is false and the other is not true, then the false watched literal has level equal to the current level.

(iv) Every implied literal appears in `to_propagate`*.*

Case (iii) occurs when during (or at the start of) UNIT-PROPAGATION a literal ℓ is implied by a clause C but we have not yet popped ℓ^C from `to_propagate`.

Note that these conditions hold after INITIALIZATION, allowing us to apply Lemma 3 for the first iteration of Algorithm 1. Lemma 3 can be proven by assuming the conditions hold at the beginning of iteration i of the while loop in UNIT-PROPAGATION, and showing that they continue to hold at the beginning of iteration $i+1$ (assuming we did not exit UNIT-PROPAGATION on line 12). One has to consider all possible cases of the states of the watched literals and what happens when one of the watched literal turns from undefined to 0/1. One also has to keep in mind Remark 2 and that `to_propagate` will be of one of the three types given by the cases (TP1), (TP2), (TP3) at the beginning of UNIT-PROPAGATION (as we will see later).

As a corollary to part *(iv)* of Lemma 3 we obtain the following lemma.

Lemma 4. UNIT-PROPAGATION *continues until a clause is falsified or until there are no more unit clauses.*

Using Lemma 3, we can then show what each clause must look like after we exit UNIT-PROPAGATION because of a conflict.

Lemma 5. *Suppose the conditions in Lemma 3 hold at the beginning of each iteration of the while loop in* UNIT-PROPAGATION. *If* UNIT-PROPAGATION *ends with a falsified clause and the current level is > 0, then every clause $C \in \mathcal{C}$ satisfies Invariant 2 and one of the following.*

(i) C is non-false, non-unit, at most one watched literal is false, and if one watched literal is false and the other is not true, the false watched literal has level equal to the current level.

(ii) C is false, has at least two literals and both watched literals are false at the current level.

(iii) C is a unit, one watched literal is undefined and the other is false at the current level.

The latter part of case (i) can occur when a watched literal was set to false but UNIT-PROPAGATION exited on line 12 before a new watched literal was selected.

Lemma 5 can be proven by considering what are all the possible states of the watched literals as we exit UNIT-PROPAGATION on line 12 by using Lemma 3. Note that by Lemma 2, all falsified clauses must have at least two literals if we falsify a clause at level > 0.

We then come CLAUSE-LEARNING with a falsified clause that has at least two literals with the current level.

Lemma 6. *Let d be the current level at the beginning of* CLAUSE-LEARNING. *Suppose at the beginning of* CLAUSE-LEARNING *the input clause C contains at least two level d literals. The while loop in* CLAUSE-LEARNING *eventually stops and when it stops the clause $C_{learned}$*

(1) is false with the current trail, and

(2) contains exactly one level d literal.

Proof. Suppose on line 3 of CLAUSE-LEARNING we have $C_{\text{learned}} = \{\ell'_1, \ldots, \ell'_k, \ell_1, \ell_2\}$ where $k \geq 0$, ℓ_1 and ℓ_2 are level d literals and C_{learned} is false. Thus the negation of every literal in C_{learned} appears in the trail. Suppose the negation of $\ell_2 \in C_{\text{learned}}$

appears latest in the trail among the negations of literals in C_{learned}. Since ℓ_1 is also a level d literal, $\neg\ell_2$ must be an implied literal. Let $R = \{h_1, \ldots, h_p, \neg\ell_2\} \in \mathcal{C}$ be the reason clause for $\neg\ell_2$. This means that $\neg\ell_2$ was implied by R at some point, and hence the literals $\neg h_1, \ldots, \neg h_p$ appear in the trail earlier than $\neg\ell_2$. The new value of C_{learned} on line 6 is then

$$\text{Res}(C_{\text{learned}}, R, \ell) = \{\ell'_1, \ldots, \ell'_k, \ell_1, h_1, \ldots, h_p\}.$$

From this we see the following.

- The value $\max\{\text{pos}(\neg\ell) \mid \ell \in C_{\text{learned}}\}$ strictly decreases on line 6.
- The new value of C_{learned} is also falsified, that is, the negations of every literal in C_{learned} appears in the trail.
- The new value of C_{learned} contains at least one level d literal.

The first bullet point shows that eventually the while loop must end. The second bullet point shows that C_{learned} stays falsified throughout the while loop and hence after the while loop ends, C_{learned} is falsified. The third bullet point shows that throughout the while loop C_{learned} has at least one level d literal in it, and thus when the algorithm stops, C_{learned} will contain exactly one level d literal in it. □

After CLAUSE-LEARNING we then add the learned clause C_{learned} to $\mathcal{C}$. The next step is to execute BACKTRACK. There we set the watched literals for C_{learned} and backtrack to some previous level. Using Lemma 5 one can show that after backtracking, there are no falsified clauses and no units other than C_{learned}, whose implied literal is in `to_propagate` (giving us case (TP1) or (TP2)). Moreover, since Invariant 2 holds for every clause before backtracking by Lemma 5, this guarantees that after backtracking every clause will satisfy Invariant 1 and Invariant 2 as we start the next unit propagation. All these remarks show that the conditions in Lemma 3 hold as we start the next unit propagation.

If UNIT-PROPAGATION does not falsify any clauses, then we choose an unassigned literal ℓ and set `to_propagate` $:= (\ell^{\text{dec}})$. Note that by Lemma 3, the conditions of the lemma will hold after UNIT-PROPAGATION ends. In particular, there are no implied literals and thus we have the case (TP3). Hence we may apply Lemma 3 again when we start the next unit propagation.

If UNIT-PROPAGATION ends by falsifying a clause at level > 0, we know by Lemma 5 that every falsified clause will have at least two literals equal to the current level. Since by Lemma 6 the learned clause C_{learned} is false before backtracking and has exactly one literal with the current level, we see that C_{learned} is not equal to any clause in $\mathcal{C}$ as we add it to $\mathcal{C}$ on line 9 of Algorithm 1. Hence we never add anything to $\mathcal{C}$ that is already there. Since there are only finitely many clauses we can have with the given variables (and we never introduce new variables), this shows that we cannot have conflicts at level > 0 infinitely many times. Therefore Algorithm 1 must eventually stop by either assigning every variable or by reaching a conflict at level 0.

4.5. **Resolution proof.** In our implementation of the CDCL algorithm we simultaneously look for a satisfying assignment and build a resolution proof of its unsatisfiability, which we explain now.

By *proof* we mean a sequence of clauses where each clause is either a clause of the original formula φ, or is obtained by resolution from clauses earlier in the proof.

If we manage to deduce the empty clause from the clauses of φ, we know φ is not satisfiable.

There are three places where we add lines to the proof: unit propagation on level 0, clause learning and conflict on level 0. There are three invariants we want to maintain throughout:

(a) for every literal ℓ in the trail on level 0, the clause $\{\ell\}$ appears in the proof,
(b) every clause in $\mathcal{C}$ is either one of the clauses of φ or appears in the proof,
(c) every clause that appears in the proof is either a clause of φ or is obtained by resolution from clauses earlier in the proof.

Remark 3. When we are adding a clause $C \in \mathcal{C}$ to the proof, we can first check if C already appears in the proof. Assuming (b) holds at this point, we know that if C does not appear in the proof, it must be one of the clauses of the original formula φ. This allows us to write a comment next to the clause, that it is an original clause.

The invariants (a), (b), and (c) hold at the beginning when the trail and the proof are empty and $\mathcal{C}$ is the set of clauses of φ. We will now show how these continue to hold as we add lines to the proof.

Unit propagation on level 0: Suppose on level 0 we have a unit $C = \{\ell_1, \ldots, \ell_k, \ell\}$ that implies ℓ and (a), (b) and (c) hold. Thus $\neg\ell_1, \ldots, \neg\ell_k$ are in the trail at level 0 and hence appear in the proof by (a). We add ℓ to the trail making ℓ true. Then, we follow the following algorithm.

Add C to the proof if not already there. Note that by (b) we know that if C is not already there, it must be one of the original clauses of φ. For each $i = 1, \ldots, k$ add

$$\mathrm{Res}(\{\ell_i, \ldots, \ell_k, \ell\}, \{\neg\ell_i\}, \ell_i)$$

to the proof. After this loop ends, ℓ is in the proof and hence (a) continues to hold. As we did not add any clauses to $\mathcal{C}$, (b) also continues to hold. Clearly (c) also continues to hold.

Hence (a), (b), (c) continue to hold whenever we add these lines to the proof during unit propagation on level 0.

Clause learning: In the procedure CLAUSE-LEARNING, before line 6 we add the clauses C_{learned} and R to the proof if not already. After we obtain the new value of C_{learned} on line 6, we add it to the proof.

Suppose (a), (b) and (c) hold as we come to CLAUSE-LEARNING and we start the first iteration of the while loop. Before line 6 we add the clauses C_{learned} and R to the proof if not already there. During the first iteration, C_{learned} is in $\mathcal{C}$. On every iteration, R is among the clauses of $\mathcal{C}$. By (b) we know that if C_{learned} and R do not already appear in the proof, they are among the clauses of φ. On line 6 of CLAUSE-LEARNING, when we obtain the new value of C_{learned} we add it to the proof. We see that (a), (b) and (c) continue to hold as we start the second iteration of the while loop. One can now inductively show that always at the beginning of the while loop in CLAUSE-LEARNING, (a), (b) and (c) hold. Hence they hold after CLAUSE-LEARNING ends, and in particular C_{learned} appears in the proof. Thus when we add C_{learned} to $\mathcal{C}$ on line 9 of Algorithm 1, we see that (b) continues to hold. Therefore (a), (b) and (c) hold after clause learning and adding the learned clause to $\mathcal{C}$.

In the procedure BACKTRACK, we do not change the proof or the set $\mathcal{C}$. We also never remove literals from the trail that have level 0. Thus (a), (b) and (c) hold as we start the next unit propagation.

We can thus conclude that (a), (b) and (c) continue to hold throughout the algorithm until we reach a conflict on level 0.

Conflict on level 0: If a clause C is falsified at level 0, then by (a) and (c) we know that the original formula must not be satisfiable. Before we exit the algorithm on line 6 of Algorithm 1, we do the following. Write $C = \{\ell_1, \ldots, \ell_k\}$. Since C is false at level 0 we know that $\{\neg\ell_1\}, \ldots, \{\neg\ell_k\}$ appear in the proof by (a). We add C to the proof if it is not already there. Again, by (b) we know that if C does not appear in the proof, we know it must be one of the original clauses of φ. For all $i = 1, \ldots, k$ we add

$$\mathrm{Res}(\{\ell_i, \ldots, \ell_k\}, \{\neg\ell_i\}, \ell_i)$$

to the proof. Once this loop finishes, we have obtained the empty clause as the last line in our proof.

5. Sudoku

Here we explain how we construct clauses from a given sudoku problem. For a positive integer m we let $[m] = \{1, \ldots, m\}$.

Let d be a positive integer and let $n = d^2$. In the usual sudoku we have $d = 3$ and $n = 9$. For each row $i \in [n]$, each column $j \in [n]$ and value $k \in [n]$ we have a variable $x_{i,j,k}$ where the idea is that

$$x_{i,j,k} = 1 \iff \text{the square in row } i\text{, column } j \text{ has a } k \text{ in it.}$$

From the given sudoku file, we can compute a set $H \subseteq [n] \times [n] \times [n]$ where each $(i, j, k) \in H$ expresses that in the square in row i column j we need to have a k in it. In other words, these are the fixed numbers in the given sudoku problem. We can express the desired conditions with CNF formulas as follows.

- Each square has at least one number in it:

$$\bigwedge_{i,j\in[n]} \bigvee_{k\in[n]} x_{i,j,k}$$

- No square has two numbers in it:

$$\bigwedge_{i,j\in[n]} \bigwedge_{\substack{k,\ell\in[n]\\ k<\ell}} (\neg x_{i,j,k} \vee \neg x_{i,j,\ell})$$

- Every row has every number at least once:

$$\bigwedge_{i\in[n]} \bigwedge_{k\in[n]} \left(\bigvee_{j\in[n]} x_{i,j,k} \right)$$

- Every column has every number at least once:

$$\bigwedge_{j\in[n]} \bigwedge_{k\in[n]} \left(\bigvee_{i\in[n]} x_{i,j,k} \right)$$

- Every box has every number at least once:

$$\bigwedge_{b_1=0}^{d-1} \bigwedge_{b_2=0}^{d-1} \bigwedge_{k\in[n]} \left(\bigvee_{(i,j)\in B(b_1,b_2)} x_{i,j,k} \right)$$

  where

$$B(b_1, b_2) = \{(b_1 d + i, b_2 d + j) \mid i, j \in [d]\}$$

  is the set of coordinates of squares in the box in position (b_1, b_2).
- The fixed squares have the correct numbers in them:

$$\bigwedge_{(i,j,k)\in H} x_{i,j,k}$$

Taking the conjunction of all these formulas, we obtain a formula that is satisfiable if and only if the given sudoku is solvable. If we find a satisfying assignment we can construct the solution to the sudoku from it. Namely, if v is a satisfying assignment then in row i column j we put the number k if and only if $v(x_{i,j,k}) = 1$.